

Packablock Readiness Checklist
Deterministic Guardrails for Tamper-Proof Software Supply Chains

Regulatory Alignment: EU Cyber Resilience Act (CRA) | SLSA Level 3+ | CISA Secure Software Guidelines

Traditional software supply chain security relies on **awareness**—static SBOM snapshots and retroactive CVE alerts that notify engineering teams hours or days *after* a compromised dependency or mutated binary has entered production.

Packablock transforms compliance into active **prevention** by embedding deterministic, cryptographic guardrails directly into CI/CD pipelines. By recording every dependency transition as an append-only, human-readable transactional delta block (`packablock.yaml`) pushed out-of-band, Packablock guarantees byte-for-byte provenance and zero main-branch clutter.

The Deterministic CI/CD Guardrail Architecture

- ↓
- 1. INGESTION & LOCK GAUNILET** → Parse lockfiles, bound SemVer drift
- ↓
- 2. HERMETIC RUNNER ISOLATION** → Keyless OIDC, isolated sandbox, SHA-256 hash
- ↓
- 3. CRYPTOGRAPHIC LEDGER LINK** → Append delta, sign block, push out-of-band
- ↓
- 4. DETERMINISTIC ENFORCEMENT** → Assert $O(1)$ stream chain, verify anchors
- ↓

Implementation & Readiness Checklist

Phase 1: Ingestion & Dependency Boundary Guardrails

Prevent untrusted, floating, or unpinned dependencies from silently entering execution paths.

- **Deterministic Lockfile Ingestion (`pkablk init`):** Lockfile states (`package-lock.json`, `bun.lockb`, Cargo manifests) must be deterministically parsed into an immutable baseline during pipeline initialization.
- **SemVer Drift & "Ghost Wick" Boundary Monitoring:** Pinned targets and upper semantic ceilings are tracked to expose unpinned breaking-change risks and historical drift velocity.
- **Pre-Execution Challenge Gates:** Require incoming dependencies to validate against an authoritative registry before build scripts or pre/post-install lifecycle hooks execute.

Phase 2: Hermetic Isolation & Build Attestation

Ensure runtime binary artifacts strictly match approved source code commits and eliminate post-build tampering.

- **Sandboxed, Hardened CI Runners:** Execute builds within isolated container environments with dropped capabilities (`-no-new-privileges`) and constrained memory caps.
- **Ephemeral, Keyless Authentication (OIDC):** Eliminate long-lived static secrets in runner environments by minting short-lived OpenID Connect (OIDC) JWT tokens tied directly to provider claims.
- **Atomic Source-to-Binary Hash Binding:** Compute raw byte-for-byte SHA-256 hashes of compiled binaries immediately upon compilation and bind them cryptographically to the commit SHA and committer identity.

Phase 3: Out-of-Band Cryptographic Ledger & Data Isolation

Maintain an un-rewritable audit trail without polluting main development branches.

- **Multi-Document YAML State Streams (`packablock.yaml1`):** Capture every dependency transition as an append-only, human-readable transactional delta block rather than heavy static JSON blobs.
- **Zero Main Branch Pollution:** Ensure pipeline enforcers write ledger states strictly out-of-band to a dedicated metadata branch (`packablock-log` / `sbom-ledger`) or an OCI registry layer.
- **Constant-Memory (SO(1)\$) Stream Verification:** Implement raw-byte streaming verification to validate block indices, metadata signatures, and chain hashes linearly without loading multi-gigabyte logs into memory.
- **Frictionless Onboarding (ALME Protocol):** Onboard repositories without manual API token copy-pasting via automated cryptographic nonce challenges (`/well-known/sbom-challenge`).

Phase 4: Continuous Provenance, Time-Travel & Audit Readiness

Ensure historic accountability and zero-overhead regulatory compliance across the software lifecycle.

- **Deterministic Time-Travel Verification (`pkablk checkout`):** Reconstruct exact byte-for-byte replicas of past configurations at any arbitrary timestamp via forward-replay state machines:


```
pkablk checkout package.json --time 2026-06-05T12:00:00Z --stdout
```
- **Genesis Rollover & Log Pruning (`pkablk rollover`):** Automatically seal, archive, and rollover historical chains when threshold limits are reached, inheriting terminal hash anchors to sustain sub-millisecond verification speeds.
- **Tiered Trust Policy Enforcement:** Enforce verification policies across standalone offline runs (P2P / degraded trust warnings) and cloud-anchored registries.
- **Automated Supply Chain Health Digests:** Deliver periodic reports summarizing verified blocks, broken chains, P2P warnings, and rollover checkpoints to maintainers.

Preparedness Matrix: Traditional Approach vs. Packablock

Compliance Vector	Legacy / Static Approach	Packablock Deterministic Guardrail
SBOM Architecture	Post-build static SPDX/CycloneDX exports generated after compilation.	Transactional Delta Chains: Live multi-doc YAML streams tracking changes linearly.
Tamper Defense	Passive CVE matching alerts hours or days after ingestion.	Preventative Gating: Pipeline halts immediately if runner or lockfile hashes diverge.
Git Cleanliness	Heavy, multi-megabyte JSON blobs committed directly to main.	Out-of-Band Routing: Isolated metadata branches (packablock-log) or OCI layers.
Audit & Provenance	Speculative git diffs over thousands of manifest lines.	Deterministic Time-Travel: Reconstruct exact historical states at any timestamp.
Secret Management	Long-lived static API tokens pasted into repository secrets.	Keyless ALME & OIDC: Ambient runner authentication via nonce challenges.

Production CI/CD Enforcer Workflow

Drop this zero-friction, hardened workflow file directly into your repository to activate automated supply chain guardrails:

```
# .github/workflows/packablock-enforcer.yml
name: Packablock Deterministic Supply Chain Enforcer

on:
  pull_request:
    paths:
      - '**/package.json'
      - '**/package-lock.json'
      - '**/bun.lockb'
  push:
    branches: [ main ]

permissions:
  contents: read
  id-token: write

jobs:
  enforce-guardrails:
    runs-on: ubuntu-latest
    container:
      image: ghcr.io/packablock/hardened-enforcer:v1
      options: --no-new-privileges

    steps:
      - name: Checkout Source
        uses: actions/checkout@v4

      - name: Verify Authoritative Ledger Anchor
        run: pkablk verify --server https://packablock.com

      - name: Evaluate Dependency Deltas & Append Out-of-Band
        run: |
          pkablk append \
            --target-file package-lock.json \
            --out-of-band-branch packablock-log

      - name: Perform Build & Attestation Check
        run: |
          bun run build
          pkablk pack --assert-clean
```

CLI Reference & Quick Start Guide

Command	Lifecycle Role	Operational Action
pkablk init	Genesis Baseline	Parses lockfiles and creates root block in packablock.yaml.
pkablk append	Delta Logging	Evaluates dependency diffs, signs delta block, appends to chain.
pkablk verify	Stream Integrity Audit	$O(1)$ memory stream check against central anchor or local P2P.
pkablk checkout	Time-Travel Replay	Reconstructs historic state at <code>-time <timestamp></code> to stdout/file.
pkablk pack	Artifact Bundling	Compiles code + chain + manifest + signature into verified tarball.
pkablk rollover	History Pruning	Seals saturated chain with tombstone block, initializes fresh chain.

- **Install CLI Engine:** `bun add -g @packablock/client` or `npm install -g packablock`
- **Initialize Baseline:** `pkablk init -target package-lock.json`
- **Verify Stream:** `pkablk verify -server https://packablock.com`